SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	H.BHARANI	**Reg. No.:**	192524385
Date:	22/10/2026	**Duration:**	60 minutes

Code of Conduct

I H. BHARANI [192524385] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

SCENARIO-BASED REQUIREMENT ANALYSIS

Personal Expense and Budget Management System

1. Study and Analyze the Scenario

1.1 Introduction

The Personal Expense and Budget Management System is an application designed to help individuals organize their income, expenses, and savings. People often make purchases throughout the day using cash, cards, or digital payments, but they may not remember every transaction. Without proper records, it becomes difficult to understand spending habits, control unnecessary purchases, or plan a monthly budget.

The proposed application provides a central place to record income and expenses, group spending into categories, set budgets, receive alerts, and view financial reports. For example, a student receiving ₹10,000 per month can record food, travel, education, and shopping expenses to see how much money remains and where spending can be reduced.

1.2 Business Domain

The system belongs to the personal finance management domain. Its purpose is not to provide banking or investment services, but to help users keep accurate records and make informed everyday spending decisions.

Aspect	Description
Domain	Personal finance and budgeting
Target users	Students, employees, families, and individuals
Inputs	Income, expenses, dates, categories, and budget limits
Processing	Balance calculation, category totals, budget comparison, and report generation
Outputs	Remaining balance, alerts, charts, and financial summaries

1.3 Existing Problems

Manual record keeping can be time-consuming and may lead to missing or incorrect entries. Users may also notice overspending only at the end of the month. The system addresses these issues by keeping transactions together and calculating totals automatically.

Problem	How the system helps
Forgotten transactions	Provides a convenient way to record transactions.
Overspending	Compares spending with budgets and shows alerts.
Unclear spending habits	Groups expenses by category.
Manual calculations	Calculates income, expenses, and balances.

1.4 Stakeholders

- Registered user: Records income and expenses, creates budgets, and views reports.
- Administrator: Maintains application settings and handles authorized technical tasks.
- Developer: Designs, implements, tests, and updates the application.
- Notification service: Delivers alerts when email or push notifications are configured.

1.5 Objectives

1. Maintain a clear record of income and expenses.
2. Categorize spending for easier analysis.
3. Allow users to set monthly and category-wise budgets.
4. Calculate the available balance automatically.
5. Notify users when spending reaches a selected threshold.
6. Generate understandable financial reports and charts.
7. Protect users' personal financial information.

2. Functional Requirements

Functional requirements describe the operations the application must perform. The following are the main features required for the proposed system.

Requirement	Description
Account management	Allow users to register, log in, and log out.
Income recording	Save the amount, date, source, and optional description.
Expense recording	Save the amount, date, category, payment method, and description.
Transaction editing	Allow users to correct or delete their own entries.
Expense categorization	Organize expenses into food, travel, education, rent, bills, and shopping.
Budget creation	Set an overall monthly limit or separate category limits.
Balance calculation	Calculate total income minus total expenses for a selected period.
Budget alerts	Warn users when spending reaches a configured threshold or exceeds the limit.
Financial reports	Display daily, monthly, or yearly summaries and category totals.
Search and export	Find transactions by date or category and export reports in supported formats.

2.1 Example Calculation

Suppose a student receives ₹10,000 in one month and spends ₹3,000 on food, ₹2,000 on travel, ₹1,500 on education, and ₹2,000 on other expenses.

Financial activity	Amount
Total income	₹10,000
Total expenses	₹8,500
Remaining balance	₹1,500

Formula: Remaining Balance = Total Income − Total Expenses. Therefore, ₹10,000 − ₹8,500 = ₹1,500. This calculation assumes all relevant transactions have been entered correctly.

2.2 Budget Alert Example

If the user sets a food budget of ₹3,000 and chooses an alert threshold of 80%, the system calculates the threshold as ₹3,000 × 0.80 = ₹2,400. When eligible food expenses reach or exceed ₹2,400, the system displays a warning. If spending is ₹2,600, the remaining budget is ₹400.

3. Non-Functional Requirements

Non-functional requirements describe the quality of the system, including how quickly it responds, how securely it handles information, and how easily users can operate it. The figures below are proposed targets to verify during testing.

Requirement	Expected quality
Performance	Common dashboard and transaction operations should respond within 2 seconds under normal conditions.
Security and privacy	Users can access only their own records; passwords are stored securely using password hashing.
Reliability	Transactions and calculations should be saved accurately without silent loss or duplication.
Usability	Forms should have clear labels, readable amounts, and helpful error messages.
Availability	Target 99.5% monthly availability, excluding agreed maintenance.
Maintainability	Use modular code so budgets, reports, and transactions can be updated independently.
Compatibility	Support common modern desktop and mobile browsers.

These targets should be measured through testing rather than assumed to be achieved automatically.

4. Actors and Use Cases

An actor is a person or external service that interacts with the application. A use case represents a goal that the actor completes through the system.

Actor	Main interactions
Registered user	Register/login, record income, record expenses, manage budgets, receive alerts, view and export reports.
Administrator	Maintain application settings and perform authorized support tasks.
Notification service	Deliver configured budget notifications.

4.1 Use Case Diagram

Diagram description: Draw a rectangle labelled “Personal Expense and Budget Management System.” Place the use cases inside it: Register/Login, Record Income, Record Expenses, Set Budget, Receive Alerts, View Reports, and Export Reports. Place the Registered User outside the rectangle and connect it to the user-related use cases. Connect the Administrator to Maintain Application and the Notification Service to Receive Alerts.

4.2 Use Case Description: Record Expense

Actor: Registered user. Precondition: The user is logged in. Main flow: The user selects Add Expense, enters the amount, date, category, and description, and submits the form. The system validates the fields and saves the transaction. Alternative flow: If a required field is missing or the amount is invalid, the system displays an error and asks the user to correct it. Postcondition: The expense is stored and included in the relevant balance, budget, and report calculations.

5. Software Requirement Specification (SRS)

5.1 Problem Description

People often maintain financial records in notebooks, separate applications, or memory. These methods can make it hard to calculate total spending and prepare monthly summaries. The proposed system brings transaction records into one application and provides automatic calculations, budget monitoring, and reports.

5.2 Scope of the System

The first version includes account access, manual transaction entry, categories, budgets, alerts, transaction history, and reports. Direct bank transfers, automatic bank synchronization, investment trading, loan approval, and tax filing are outside the initial scope. These may be considered in later versions if the project has suitable resources and security arrangements.

5.3 Data Requirements

Data entity	Examples of stored information
User	User ID, name, email, password hash, preferences
Income	Income ID, amount, date, source, user ID
Expense	Expense ID, amount, date, category, payment method, user ID
Budget	Budget period, category, spending limit, alert threshold
Notification	Alert type, date, user ID, delivery status

Amounts must be positive, dates must be valid, and each transaction must be associated with the correct user. When a transaction is edited or deleted, relevant totals and budget status must be recalculated.

5.4 External Interface Requirements

- User interface: Dashboard, transaction forms, budget screens, and reports.
- Database interface: Stores and retrieves financial records consistently.
- Notification interface: Connects to a supported service if external alerts are enabled.
- Browser interface: Provides a responsive layout for desktop and mobile devices.

6. Assumptions and Constraints

6.1 Assumptions

- Users have access to a suitable device and internet connection.
- Users enter transaction information accurately and regularly.
- Users create budgets based on their personal needs.
- A default set of expense categories is available.

6.2 Constraints

- Technical: The design depends on the selected programming language, framework, and database.
- Economic: Hosting and notification services may have costs.
- Time: Development and testing must fit the academic deadline.
- Operational: Manual entry means reports may be incomplete if users forget transactions.
- Security and privacy: Personal records must be protected and applicable privacy requirements followed.

7. Requirement Validation

Requirement validation checks whether the requirements are clear, complete, consistent, feasible, and testable. The following test cases can be used to verify the main features.

Test scenario	Expected result
Record ₹10,000 income	Income is saved and included in the total.
Record a ₹500 food expense	Expense is saved under Food.
Calculate balance with ₹8,500 expenses	Remaining balance is ₹1,500.
Food budget ₹3,000; threshold 80%; spending ₹2,500	Alert is triggered because spending exceeds ₹2,400.
Enter a negative expense	System rejects the invalid amount.
Open a monthly report	The report shows correct transactions and totals for that month.
Attempt to view another user's records	Access is denied.

7.1 Improvements

The requirements can be improved by defining exactly when alerts repeat, using consistent date ranges in reports, validating all inputs, protecting records on the server, and testing calculation accuracy. Future versions may support recurring expenses, savings goals, or secure bank imports.

8. Proposed System Architecture

A three-layer architecture is suitable for this project:

- Presentation layer: Displays the dashboard, forms, budget status, and reports.
- Application layer: Validates entries, calculates balances, checks budgets, triggers alerts, and generates reports.
- Data layer: Stores user accounts, transactions, categories, budgets, and notification status.

Separating these layers makes the application easier to test and maintain. For example, report calculations can be changed without redesigning the entire interface.

9. Expected Outcomes

- Income and expenses are organized in one place.
- Financial balances are calculated automatically.
- Users can understand category-wise spending.
- Budget alerts help users notice overspending earlier.
- Reports support monthly review and future planning.
- Account-based access helps protect private financial information.

10. Conclusion

The Personal Expense and Budget Management System offers a practical way to organize personal finances. It allows users to record income and expenses, categorize spending, set budgets, receive alerts, and review financial reports. This requirement analysis identifies the main system features, quality expectations, actors, data needs, assumptions, constraints, and validation tests. If implemented and tested carefully, the application can help users understand their spending and make more informed financial decisions.

11. Detailed System Workflow

The system workflow describes how information moves through the application from the time a user enters a transaction until the dashboard and reports are updated. A clear workflow helps developers implement the same rules consistently across different screens.

- Step 1 – Login: The user signs in with valid credentials. The system checks the account and opens the user's dashboard.
- Step 2 – Enter transaction: The user chooses Add Income or Add Expense and enters the amount, date, category, and other required details.
- Step 3 – Validate data: The system checks that required fields are present, the amount is valid, and the transaction belongs to the signed-in user.
- Step 4 – Save record: If the information is valid, the application stores the transaction in the database.
- Step 5 – Update calculations: The application recalculates the relevant income total, expense total, available balance, and category budget usage.

- Step 6 – Check alerts: The system compares eligible spending with the user's configured budget thresholds and creates an alert if a threshold is reached.
- Step 7 – Display results: The dashboard and reports show the updated figures so the user can review their financial position.

If a transaction is edited or deleted, the system must recalculate the affected totals. This prevents the dashboard from displaying outdated values.

12. User Interface and Dashboard Requirements

The dashboard is the main screen users see after logging in. It should present the most useful financial information without requiring users to open many separate pages. A simple layout is especially important for users who may not have experience with budgeting applications.

- Summary cards should display total income, total expenses, remaining balance, and current budget usage.
- A quick-action area should provide buttons such as Add Income, Add Expense, and Set Budget.
- A category chart should help users compare spending on food, travel, education, bills, and other categories.
- A recent-transactions section should display the latest entries with date, category, description, and amount.
- A budget-status section should show the budget limit, amount spent, amount remaining, and any active alerts.
- Forms should explain errors in plain language, such as “Enter an amount greater than zero” rather than displaying technical error messages.
- The layout should adapt to smaller screens so users can record expenses from a phone.

14. Security, Privacy, and Data Protection

Financial records are private and may reveal information about a user's daily activities. Therefore, security must be considered during design and implementation, not added only after the application is completed.

- Authentication: Require users to sign in before viewing personal financial information.
- Authorization: Check on the server that a user can access only their own transactions, budgets, and reports.
- Password protection: Store passwords using a suitable password-hashing algorithm rather than plain text.
- Secure communication: Use HTTPS when the application is deployed online.
- Input validation: Validate data on the server even when the browser already checks the form.
- Session safety: End or expire inactive sessions according to the application's security policy.
- Data backup: Maintain protected backups and test whether records can be restored.
- Data minimization: Collect only information needed for the application's functions and explain how it is used.

These measures reduce the risk of unauthorized access and accidental data loss. A student prototype should still follow basic security practices even if it does not connect to real bank accounts.

15. Acceptance Criteria

Acceptance criteria describe the conditions that must be met before a feature can be considered complete. They help the developer and evaluator agree on what successful implementation means.

- A user can create an account and sign in with valid credentials.
- A valid income or expense entry is saved and appears in the transaction history.
- Invalid amounts and missing required fields are rejected with understandable messages.
- The dashboard balance matches total recorded income minus total recorded expenses for the selected period.
- A budget alert appears when spending reaches the configured threshold.
- Editing or deleting a transaction updates the balance, category totals, and budget status.
- A user cannot view or modify another user's financial records.
- The main screens remain usable on both desktop and mobile browser sizes.

16. Future Enhancements

The first version should focus on reliable manual transaction entry, budgets, alerts, and reports. After these features are stable, additional functions can be considered based on user feedback and available resources.

- Recurring transactions: Automatically prepare entries for regular expenses such as rent or subscriptions, while allowing the user to confirm them.
- Savings goals: Let users set a target amount and monitor progress toward it.
- CSV import: Allow users to import transaction files after validating their format and contents.
- Secure bank integration: Consider importing transactions through a supported and authorized provider, subject to privacy, security, and legal requirements.
- Custom categories: Allow users to create categories that match their personal spending habits.
- Multi-currency support: Provide consistent handling of transactions recorded in different currencies if this becomes a project requirement.
- Reminder settings: Allow users to choose suitable reminders for recording expenses or reviewing a monthly budget.